VRUSHABH SAWANT SYVLSI_44 125BTVL1033

Experiment 4

Academic Year 2026-2027			
Program: Electronics Engineering (VLSI Design and Technology)			
Year of Study	Second Year	**Semester**	III
Course	Skill Lab: Python Programming	**Course Code**	VDCOR2VS201
Class	SY VLSI	**Course In-Charge**	Mr. Nirmol Munvar
Exp Name	Control Structures in Python		
CO	Analyze and implement control structures and user-defined functions to solve logical and numerical problems using Python.		

Control Structures in Python

Control structures determine the **flow of execution** of a Python program. They allow a program to make decisions, repeat statements, and execute different blocks of code based on conditions.

if Statement

The if statement is used to execute a block of code only when a specified condition is True.

Syntax:

```
if condition:
    statement
```

Example:

```
age = 20

if age >= 18:
    print("Eligible to vote")
```

The statements inside the if block must be indented. Python commonly uses **four spaces** for indentation.

if-else Statement

The if-else structure provides two possible execution paths. If the condition is True, the if block executes; otherwise, the else block executes.

Syntax:

```
if condition:
    statement1
else:
    statement2
```

Example:

```
num = 15

if num % 2 == 0:
    print("Even number")
else:
    print("Odd number")
```

if-elif-else Statement

The if-elif-else structure is used when there are **multiple conditions** to evaluate.

Python checks the conditions from top to bottom. Once a condition is True, its corresponding block is executed and the remaining conditions are skipped.

Syntax:

```
if condition1:
    statement1
elif condition2:
    statement2
elif condition3:
    statement3
else:
    statement4
```

Example:

```
marks = 75

if marks >= 90:
    print("Grade A+")
elif marks >= 80:
    print("Grade A")
elif marks >= 70:
    print("Grade B")
elif marks >= 60:
    print("Grade C")
else:
    print("Fail")
```

There can be any number of elif blocks, but there can be only one if and one optional else associated with a particular conditional structure.

Nested if

An if statement can be placed inside another if statement. This is called a **nested if**.

Example:

```
age = 20
citizen = True

if age >= 18:
    if citizen:
        print("Eligible to vote")
```

Nested conditions are useful when one condition needs to be checked only after another condition is satisfied.

match-case Statement

The match-case statement is available in Python 3.10 and later. It is used for **pattern matching** and can be useful when a variable needs to be compared against several possible patterns or values.

It is conceptually similar to a switch-case structure found in languages such as C, C++, and Java, but Python's pattern matching is more powerful.

Basic syntax:

```
match expression:
    case value1:
        statement1
    case value2:
        statement2
    case _:
```

```
    statement3
```
The _ represents a **wildcard pattern** and acts as the default case.

Example:

```
day = 2

match day:
    case 1:
        print("Monday")
    case 2:
        print("Tuesday")
    case 3:
        print("Wednesday")
    case _:
        print("Invalid day")
```
The match expression is evaluated, and Python compares it with the patterns specified in the case statements.

match-case with Multiple Values

Multiple values can be matched using the | operator.

```
day = 6

match day:
    case 1 | 2 | 3 | 4 | 5:
        print("Weekday")
    case 6 | 7:
        print("Weekend")
    case _:
        print("Invalid day")
```
match-case with Conditions

A case can also contain a guard using if.

```
num = 15

match num:
    case n if n > 0:
        print("Positive")
    case n if n < 0:
        print("Negative")
    case 0:
        print("Zero")
```
The if after the pattern is called a **guard**. The case is selected only when both the pattern and guard condition are satisfied.

for Loop

A for loop is used to repeatedly execute a block of code for each item in an iterable such as a list, tuple, string, range, or dictionary.

Syntax:

```
for variable in iterable:
    statement
```
Example:

```
numbers = [10, 20, 30, 40]

for num in numbers:
```

```
    print(num)
```
The loop assigns each element of the list to num one at a time.

for Loop with range()

The range() function is commonly used with for loops when a specific sequence of numbers is required.

```
for i in range(5):
    print(i)
```
Output:
```
0
1
2
3
4
```
range() can accept three parameters:

range(start, stop, step)

The stop value is not included.

Example:
```
for i in range(2, 11, 2):
    print(i)
```
This prints:
```
2
4
6
8
10
```
while Loop

A while loop repeatedly executes a block of code as long as its condition remains True.

Syntax:
```
while condition:
    statement
```
Example:
```
count = 1

while count <= 5:
    print(count)
    count += 1
```
The condition is checked before every iteration. Therefore, if the condition is initially False, the loop will not execute even once.

Nested Loops

A loop can be placed inside another loop. This is called a **nested loop**.

Example:
```
for i in range(1, 4):
    for j in range(1, 4):
        print(i, j)
```
The inner loop completes all its iterations for every iteration of the outer loop.

Nested loops are commonly used for matrices, tables, patterns, and multidimensional data processing.

break Statement

The break statement immediately terminates the nearest enclosing loop.

Example:

```
for i in range(1, 10):
    if i == 5:
        break
    print(i)
```
Output:
1
2
3
4

continue Statement

The continue statement skips the remaining statements in the current iteration and moves to the next iteration of the loop.

Example:

```
for i in range(1, 6):
    if i == 3:
        continue
    print(i)
```
Output:
1
2
4
5

pass Statement

The pass statement does nothing. It is used as a placeholder where Python requires a statement syntactically, but no action is currently required.

```
if age >= 18:
    pass
else:
    print("Minor")
```
It is also useful while developing program structures that will be implemented later.

Loop with else

Python allows an else block to be associated with a for or while loop.

The else block executes when the loop finishes normally. It does **not** execute if the loop is terminated using break.

Example:

```
for i in range(5):
    print(i)
else:
    print("Loop completed")
```
With break:
```
for i in range(5):
    if i == 3:
        break
    print(i)
else:
    print("Loop completed")
```
In this case, "Loop completed" is not printed because the loop was terminated using break.

Exercises

Exercise 1: Student Grade and Result System

Write a Python program that accepts marks for **5 subjects** using a for loop. Calculate the total and percentage.

Use if-elif-else to assign grades:

- 90 and above: A+
- 80–89: A
- 70–79: B
- 60–69: C
- 50–59: D
- Below 50: F

Use a while loop to allow the user to calculate results for multiple students. Use break when the user chooses to stop.

Exercise 2: Menu-Driven Calculator

Write a menu-driven calculator using match-case. The menu should contain:

- Addition
- Subtraction
- Multiplication
- Division
- Exit

Accept two numbers from the user and perform the selected operation.

Use if-else to handle division by zero. Use a while loop to repeatedly display the menu until the user selects Exit.

Exercise 3: Number Analysis

Write a Python program that accepts **10 integers** from the user using a for loop.

For each number:

- Use if-elif-else to determine whether it is positive, negative, or zero.
- Skip negative numbers using continue.
- Calculate the sum of positive numbers.
- Use break if the user enters 999, which should immediately terminate the input process.

After the loop, display the number of positive, negative, and zero values entered.

Exercise 4: Multiplication Table Generator

Write a Python program that accepts a number from the user and generates its multiplication table from 1 to 10 using a for loop.

Then use a while loop to repeatedly ask whether the user wants to generate another table.

Use if-else to validate whether the entered number is positive.

Use break to terminate the program when the user chooses not to continue.

Exercise 5: Simple ATM System

Write a menu-driven ATM program using match-case. The menu should provide:

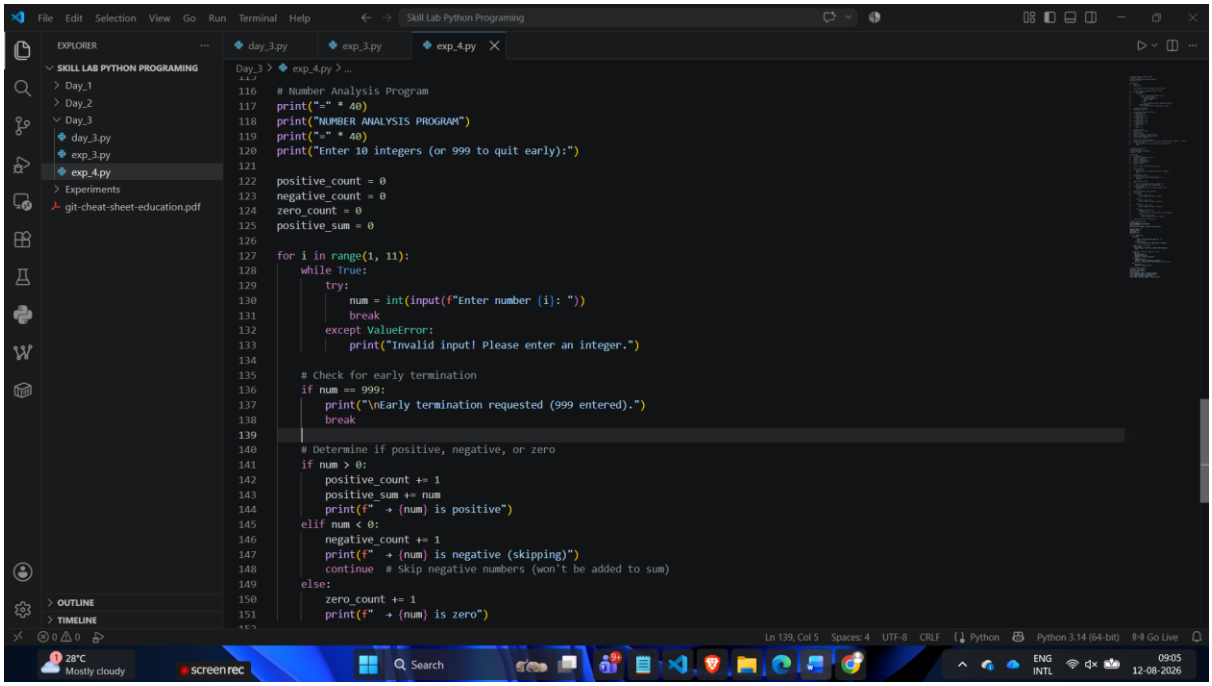
- Check Balance
- Deposit
- Withdraw
- Exit

Use a while loop to repeatedly display the menu.

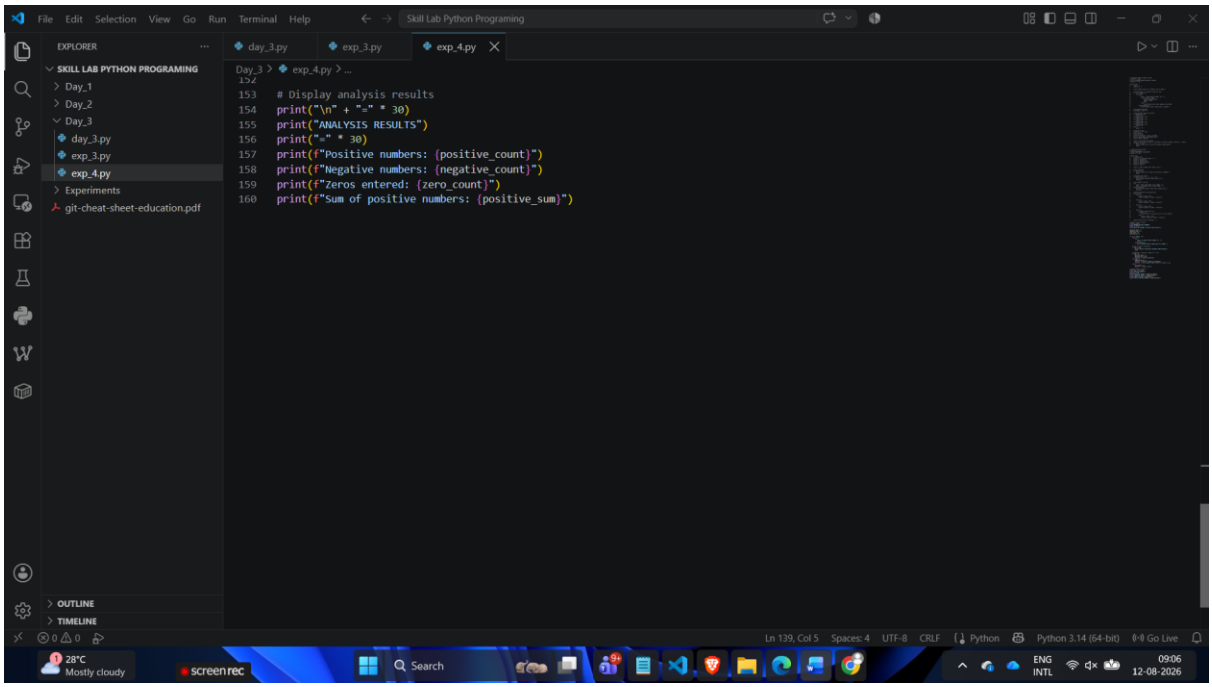
Use if-elif-else to validate withdrawal conditions:

- Withdrawal amount must be positive.
- Withdrawal amount must not exceed the available balance.
- Otherwise, display an appropriate message.

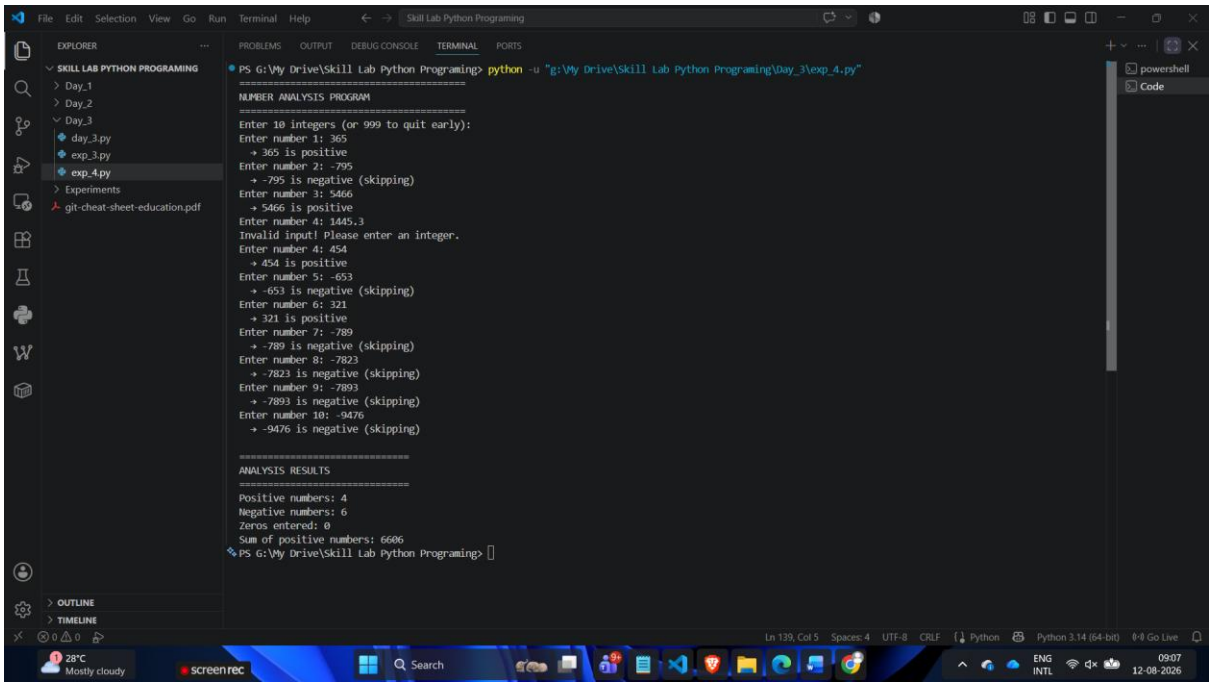
Use a for loop to allow a maximum of **3 withdrawal attempts**. Use continue when an invalid withdrawal amount is entered and break when a valid withdrawal is completed.



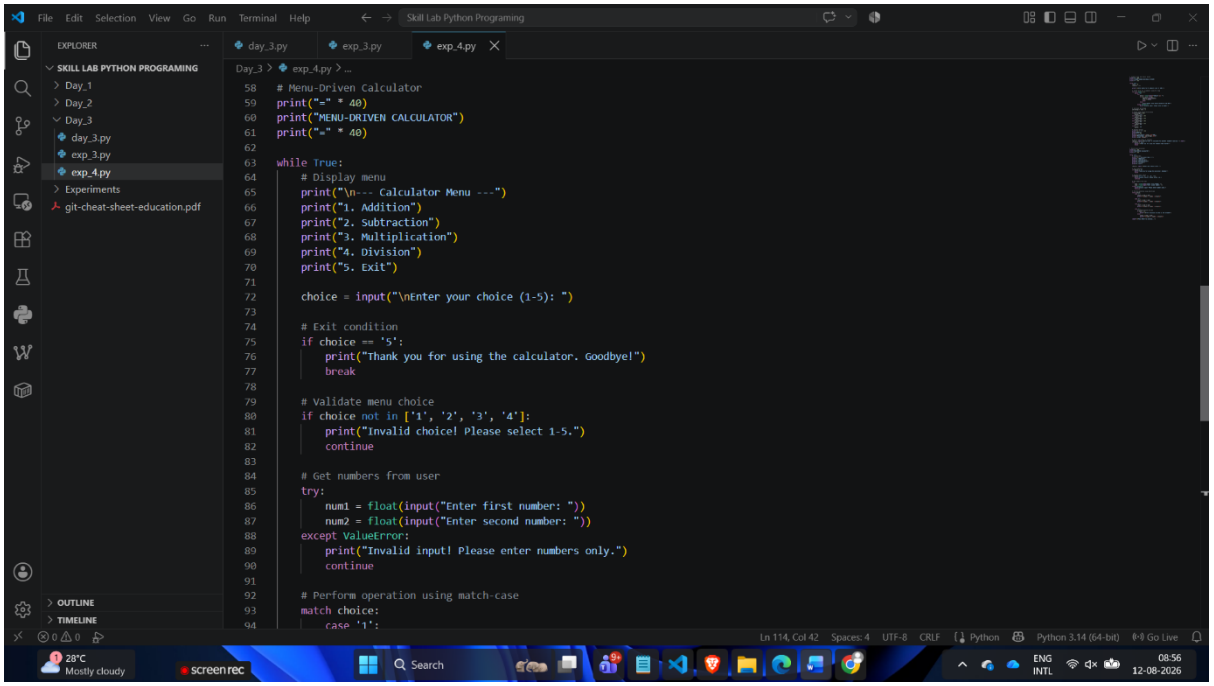
```python
# Number Analysis Program
print("=" * 40)
print("NUMBER ANALYSIS PROGRAM")
print("=" * 40)
print("Enter 10 integers (or 999 to quit early):")

positive_count = 0
negative_count = 0
zero_count = 0
positive_sum = 0

for i in range(1, 11):
    while True:
        try:
            num = int(input(f"Enter number {i}: "))
            break
        except ValueError:
            print("Invalid input! Please enter an integer.")

    # Check for early termination
    if num == 999:
        print("\nEarly termination requested (999 entered).")
        break

    # Determine if positive, negative, or zero
    if num > 0:
        positive_count += 1
        positive_sum += num
        print(f"  → {num} is positive")
    elif num < 0:
        negative_count += 1
        print(f"  → {num} is negative (skipping)")
        continue  # Skip negative numbers (won't be added to sum)
    else:
        zero_count += 1
        print(f"  → {num} is zero")
```



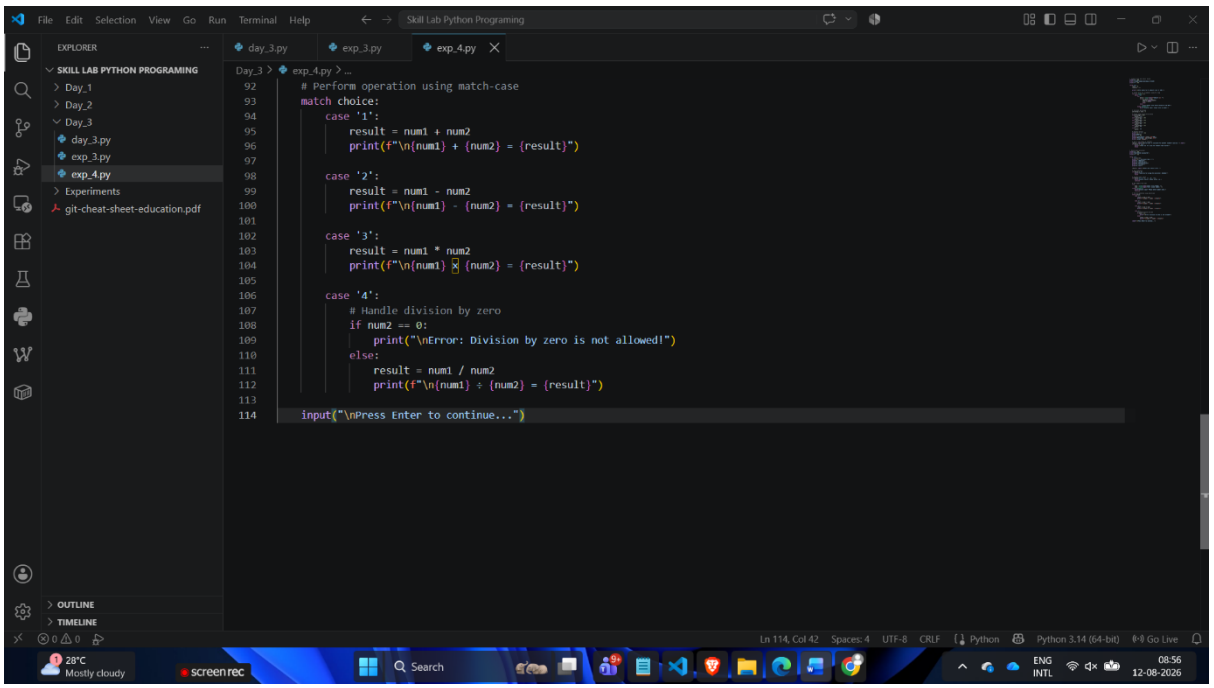
```python
# Display analysis results
print("\n" + "=" * 30)
print("ANALYSIS RESULTS")
print("=" * 30)
print(f"Positive numbers: {positive_count}")
print(f"Negative numbers: {negative_count}")
print(f"Zeros entered: {zero_count}")
print(f"Sum of positive numbers: {positive_sum}")
```



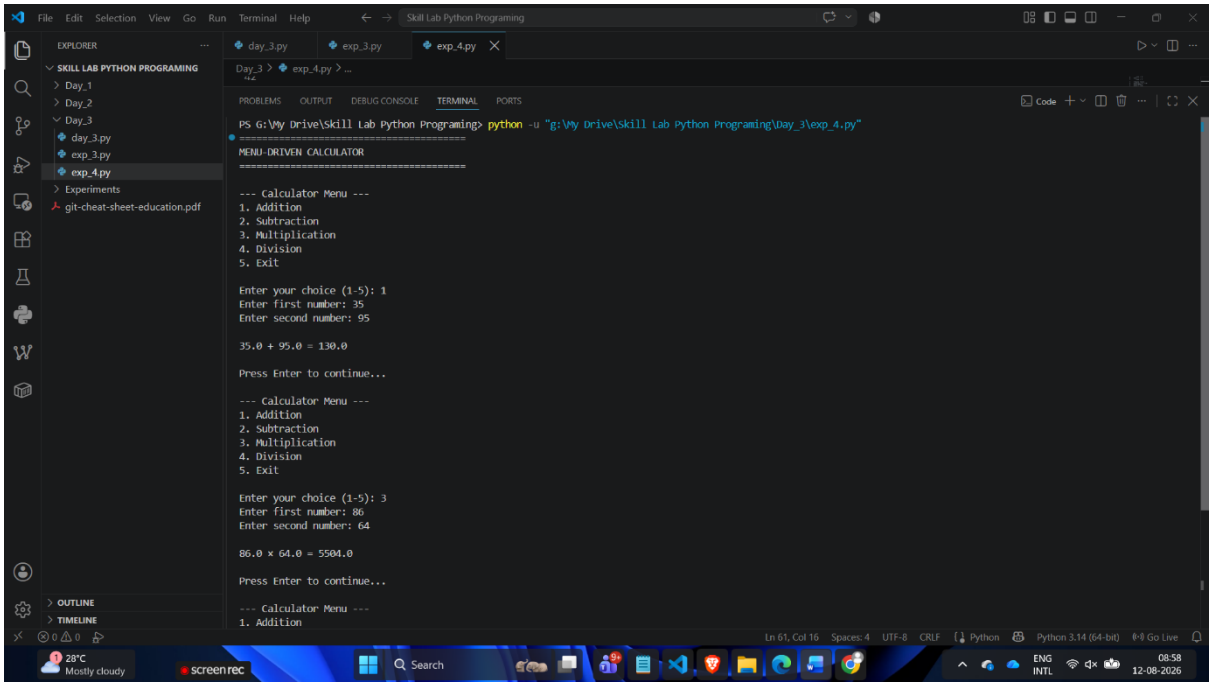
```
PS G:\My Drive\Skill Lab Python Programing> python -u "g:\My Drive\Skill Lab Python Programing\Day_3\exp_4.py"
========================================
NUMBER ANALYSIS PROGRAM
========================================
Enter 10 integers (or 999 to quit early):
Enter number 1: 365
  → 365 is positive
Enter number 2: -795
  → -795 is negative (skipping)
Enter number 3: 5466
  → 5466 is positive
Enter number 4: 1445.3
Invalid input! Please enter an integer.
Enter number 4: 454
  → 454 is positive
Enter number 5: -653
  → -653 is negative (skipping)
Enter number 6: 321
  → 321 is positive
Enter number 7: -789
  → -789 is negative (skipping)
Enter number 8: -7823
  → -7823 is negative (skipping)
Enter number 9: -7893
  → -7893 is negative (skipping)
Enter number 10: -9476
  → -9476 is negative (skipping)

==============================
ANALYSIS RESULTS
==============================
Positive numbers: 4
Negative numbers: 6
Zeros entered: 0
Sum of positive numbers: 6606
PS G:\My Drive\Skill Lab Python Programing>
```



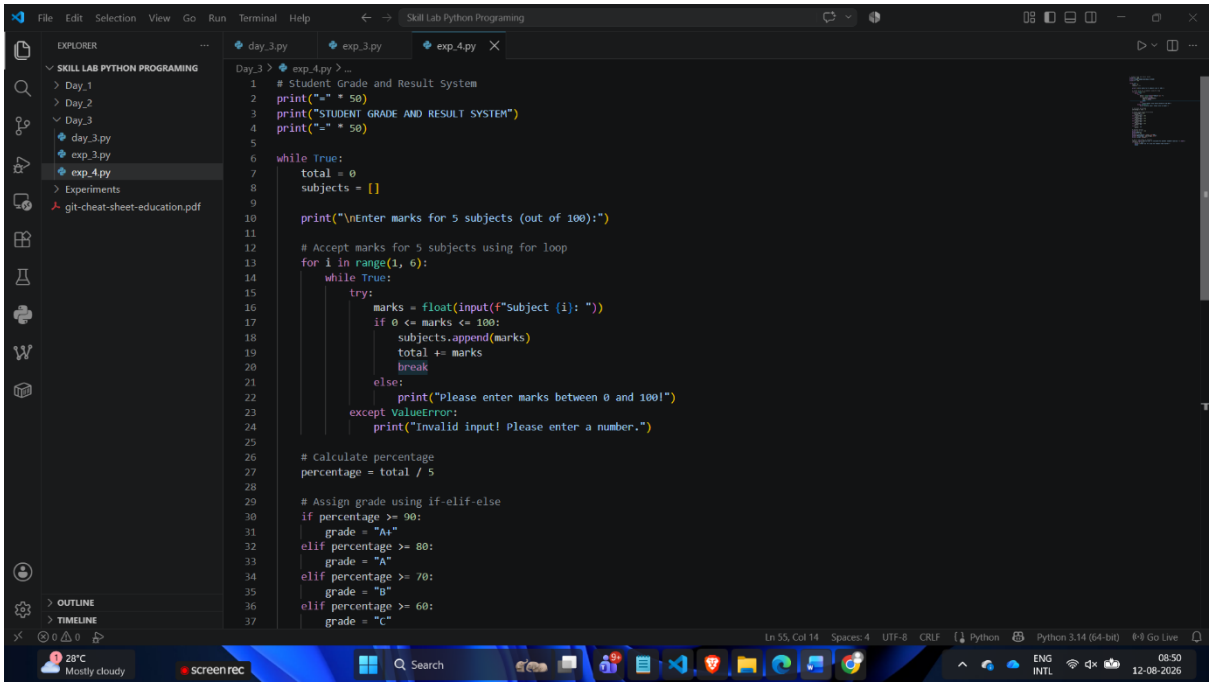
```python
# Menu-Driven Calculator
print("=" * 40)
print("MENU-DRIVEN CALCULATOR")
print("=" * 40)

while True:
    # Display menu
    print("\n--- Calculator Menu ---")
    print("1. Addition")
    print("2. Subtraction")
    print("3. Multiplication")
    print("4. Division")
    print("5. Exit")

    choice = input("\nEnter your choice (1-5): ")

    # Exit condition
    if choice == '5':
        print("Thank you for using the calculator. Goodbye!")
        break

    # Validate menu choice
    if choice not in ['1', '2', '3', '4']:
        print("Invalid choice! Please select 1-5.")
        continue

    # Get numbers from user
    try:
        num1 = float(input("Enter first number: "))
        num2 = float(input("Enter second number: "))
    except ValueError:
        print("Invalid input! Please enter numbers only.")
        continue

    # Perform operation using match-case
    match choice:
        case '1':
```



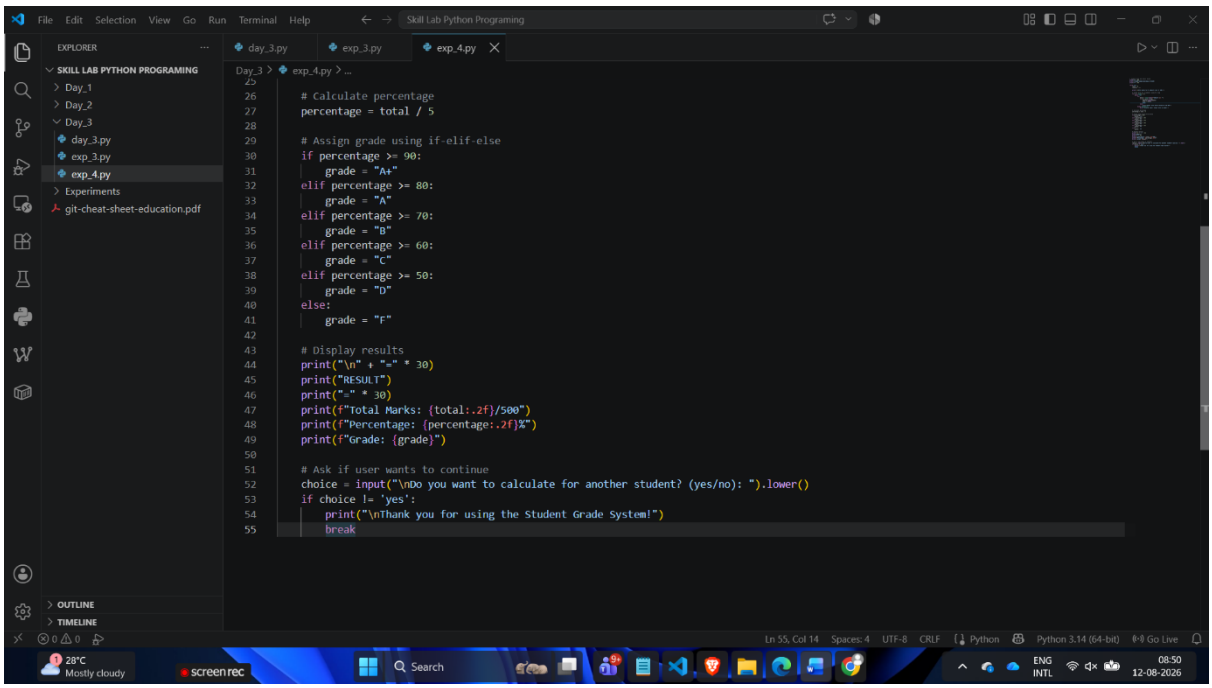
```python
    # Perform operation using match-case
    match choice:
        case '1':
            result = num1 + num2
            print(f"\n{num1} + {num2} = {result}")

        case '2':
            result = num1 - num2
            print(f"\n{num1} - {num2} = {result}")

        case '3':
            result = num1 * num2
            print(f"\n{num1} × {num2} = {result}")

        case '4':
            # Handle division by zero
            if num2 == 0:
                print("\nError: Division by zero is not allowed!")
            else:
                result = num1 / num2
                print(f"\n{num1} ÷ {num2} = {result}")

    input("\nPress Enter to continue...")
```



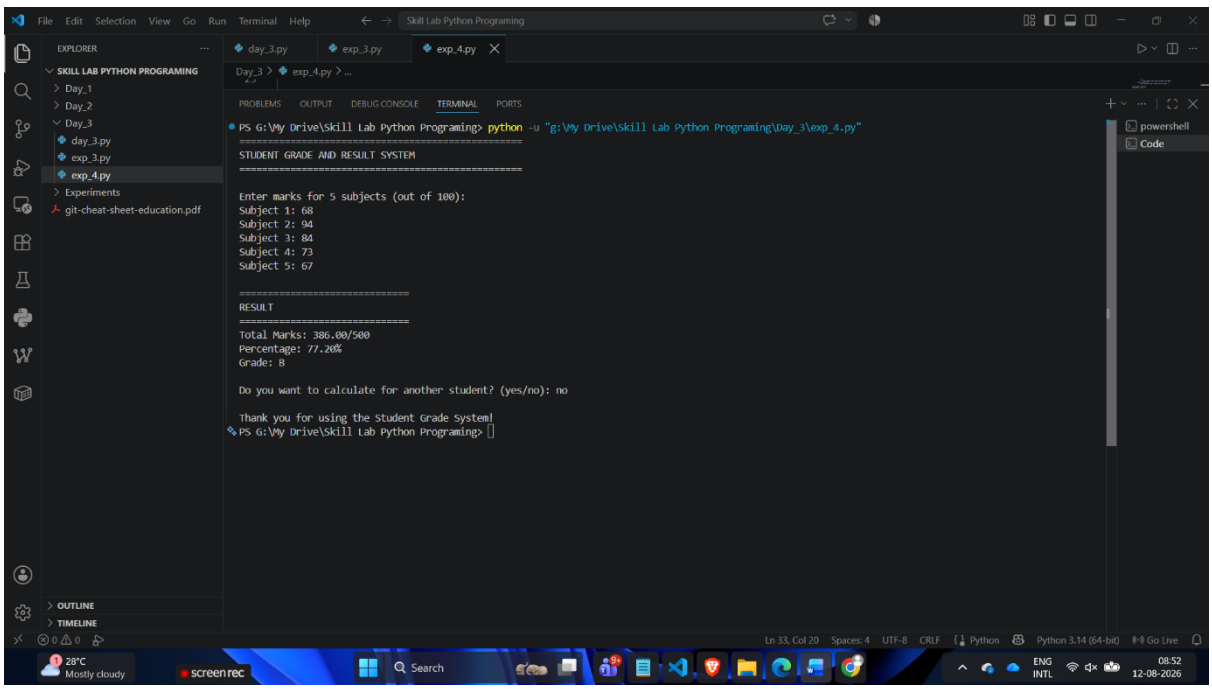
```
PS G:\My Drive\Skill Lab Python Programing> python -u "g:\My Drive\Skill Lab Python Programing\Day_3\exp_4.py"
========================================
MENU-DRIVEN CALCULATOR
========================================

--- Calculator Menu ---
1. Addition
2. Subtraction
3. Multiplication
4. Division
5. Exit

Enter your choice (1-5): 1
Enter first number: 35
Enter second number: 95

35.0 + 95.0 = 130.0

Press Enter to continue...

--- Calculator Menu ---
1. Addition
2. Subtraction
3. Multiplication
4. Division
5. Exit

Enter your choice (1-5): 3
Enter first number: 86
Enter second number: 64

86.0 × 64.0 = 5504.0

Press Enter to continue...

--- Calculator Menu ---
1. Addition
```



```python
# Student Grade and Result System
print("=" * 50)
print("STUDENT GRADE AND RESULT SYSTEM")
print("=" * 50)

while True:
    total = 0
    subjects = []

    print("\nEnter marks for 5 subjects (out of 100):")

    # Accept marks for 5 subjects using for loop
    for i in range(1, 6):
        while True:
            try:
                marks = float(input(f"Subject {i}: "))
                if 0 <= marks <= 100:
                    subjects.append(marks)
                    total += marks
                    break
                else:
                    print("Please enter marks between 0 and 100!")
            except ValueError:
                print("Invalid input! Please enter a number.")

    # Calculate percentage
    percentage = total / 5

    # Assign grade using if-elif-else
    if percentage >= 90:
        grade = "A+"
    elif percentage >= 80:
        grade = "A"
    elif percentage >= 70:
        grade = "B"
    elif percentage >= 60:
        grade = "C"
```



```python
    # Calculate percentage
    percentage = total / 5

    # Assign grade using if-elif-else
    if percentage >= 90:
        grade = "A+"
    elif percentage >= 80:
        grade = "A"
    elif percentage >= 70:
        grade = "B"
    elif percentage >= 60:
        grade = "C"
    elif percentage >= 50:
        grade = "D"
    else:
        grade = "F"

    # Display results
    print("\n" + "=" * 30)
    print("RESULT")
    print("=" * 30)
    print(f"Total Marks: {total:.2f}/500")
    print(f"Percentage: {percentage:.2f}%")
    print(f"Grade: {grade}")

    # Ask if user wants to continue
    choice = input("\nDo you want to calculate for another student? (yes/no): ").lower()
    if choice != 'yes':
        print("\nThank you for using the Student Grade System!")
        break
```



```
PS G:\My Drive\Skill Lab Python Programing> python -u "g:\My Drive\Skill Lab Python Programing\Day_3\exp_4.py"
==================================================
STUDENT GRADE AND RESULT SYSTEM
==================================================

Enter marks for 5 subjects (out of 100):
Subject 1: 68
Subject 2: 94
Subject 3: 84
Subject 4: 73
Subject 5: 67

==============================
RESULT
==============================
Total Marks: 386.00/500
Percentage: 77.20%
Grade: B

Do you want to calculate for another student? (yes/no): no

Thank you for using the Student Grade System!
PS G:\My Drive\Skill Lab Python Programing>
```